18-740: Modern Computer Architecture & Design

Lecture 3:
"Advanced Branch Prediction Techniques"

John P. Shen

September 1, 2026

- Assignment for This Week:
  - **Lab 1** will be posted by Tuesday (9/1).
- Required Reading Assignments:
  - **Chapters 5 and 9 of Shen and Lipasti (SnL)**
- Recommended Reference:
  - T. Yeh and Y. Patt, "Two-Level Adaptive Branch Prediction," Intl. Symposium on Microarchitecture, November 1991.

Electrical & Computer ENGINEERING

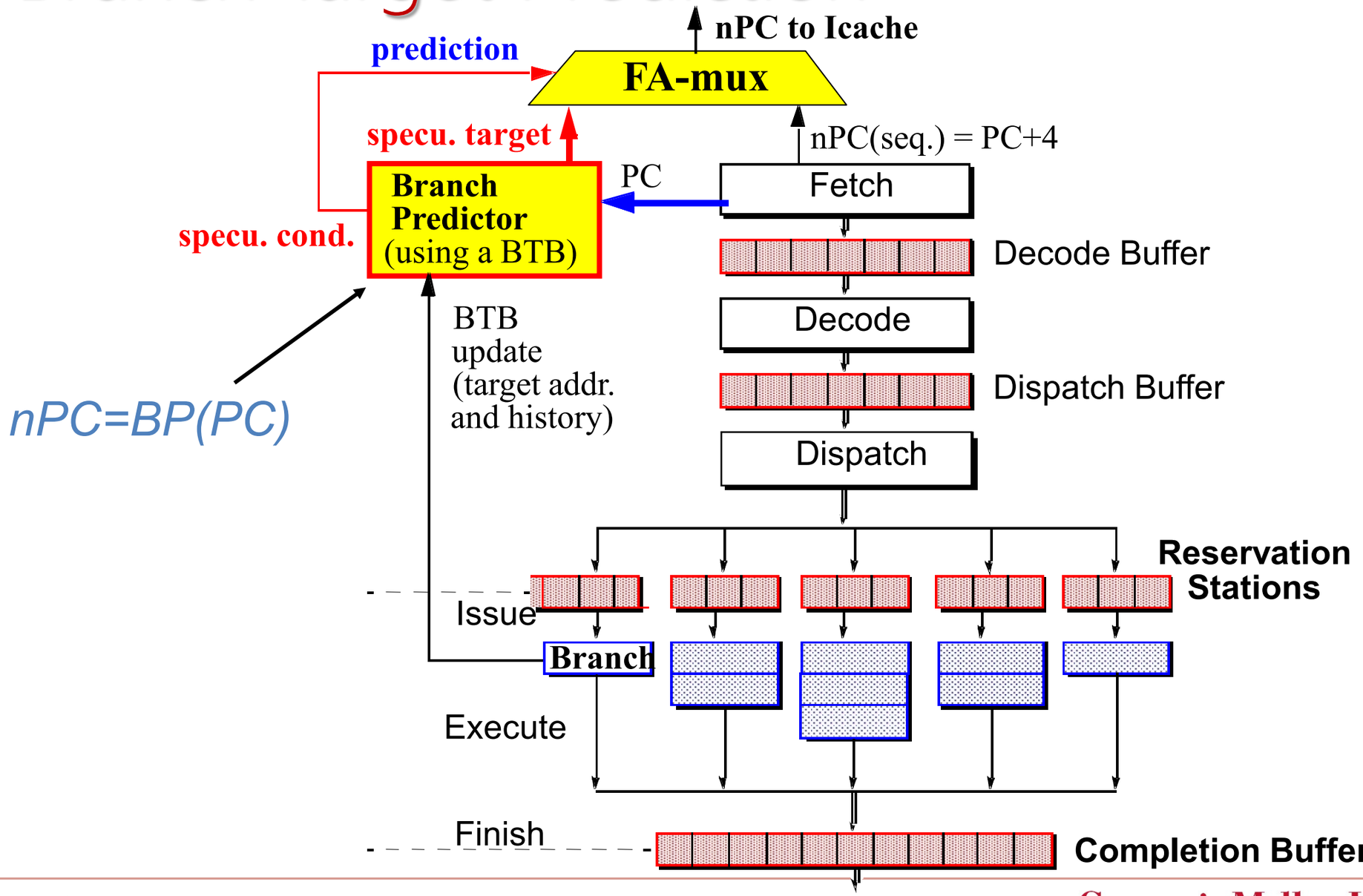
18-740: Modern Computer Architecture & Design

Lecture 3:
"Advanced Branch Prediction Techniques"

A. **Basic Dynamic Branch Prediction** (review)
B. Two-Level Adaptive Branch Predictors
C. Interference Reducing Predictors
D. Perceptron Based Predictors
E. Hybrid Branch Predictors

Electrical & Computer ENGINEERING

Branch Target Prediction

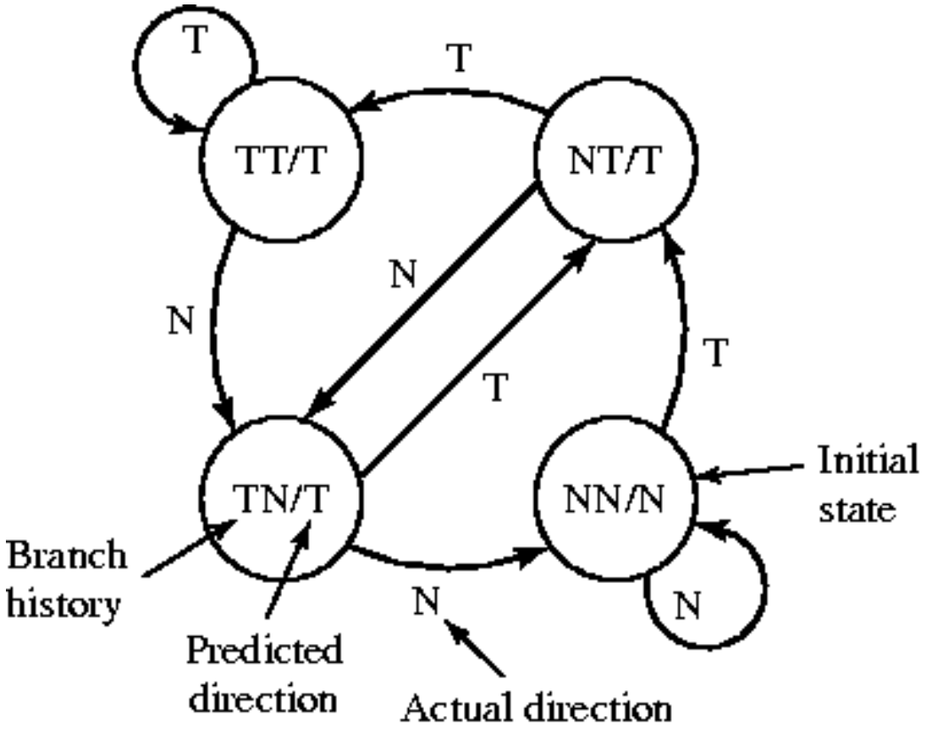


[James E Smith, CDC, 1981]

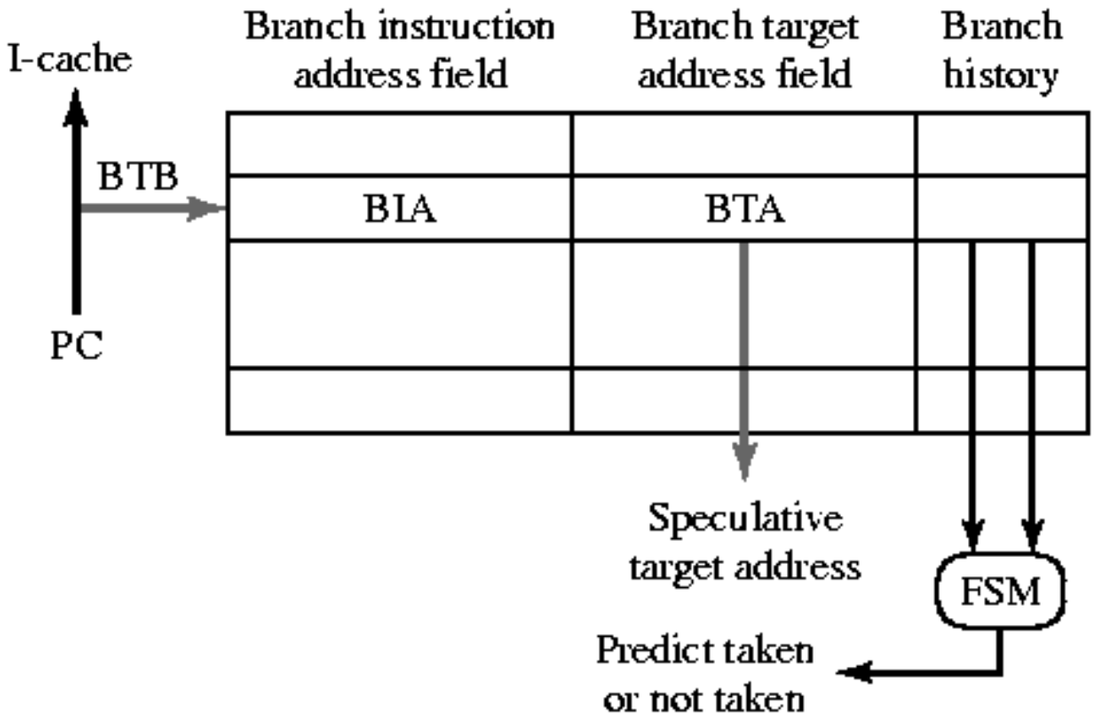
History-Based Branch Prediction



James E. Smith



(a)



(b)

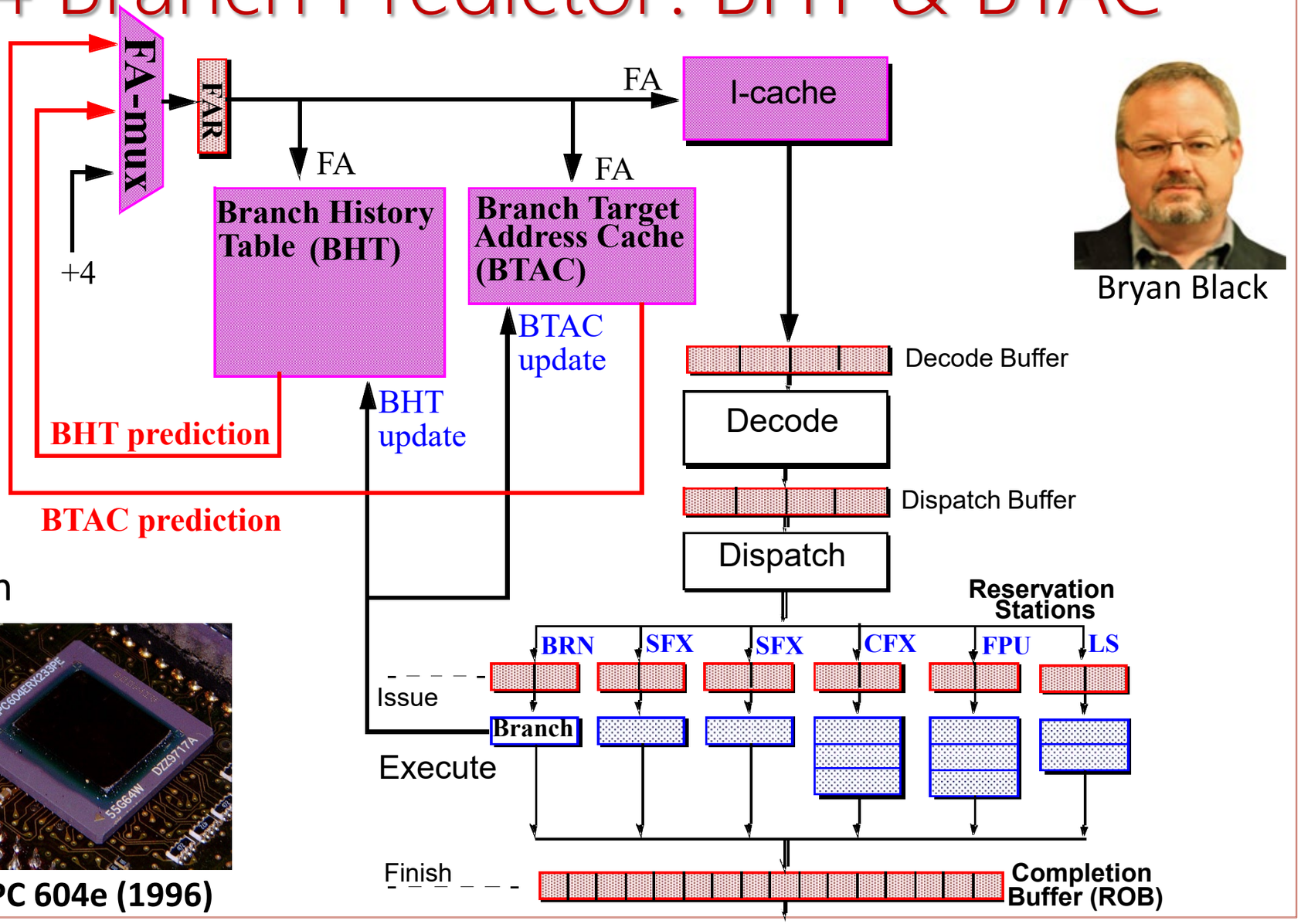
PowerPC 604 Branch Predictor: BHT & BTAC

BTAC:

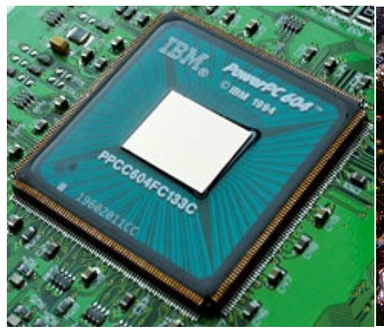
- 64 entries
- Fully associative
- Hit → predict taken

BHT:

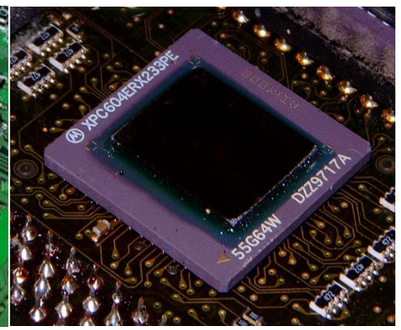
- 512 entries
- Direct mapped
- 2-bit saturating counter
- History based prediction
- Overrides BTAC prediction



Bryan Black



4-IPC 604 (1994)



6-IPC 604e (1996)

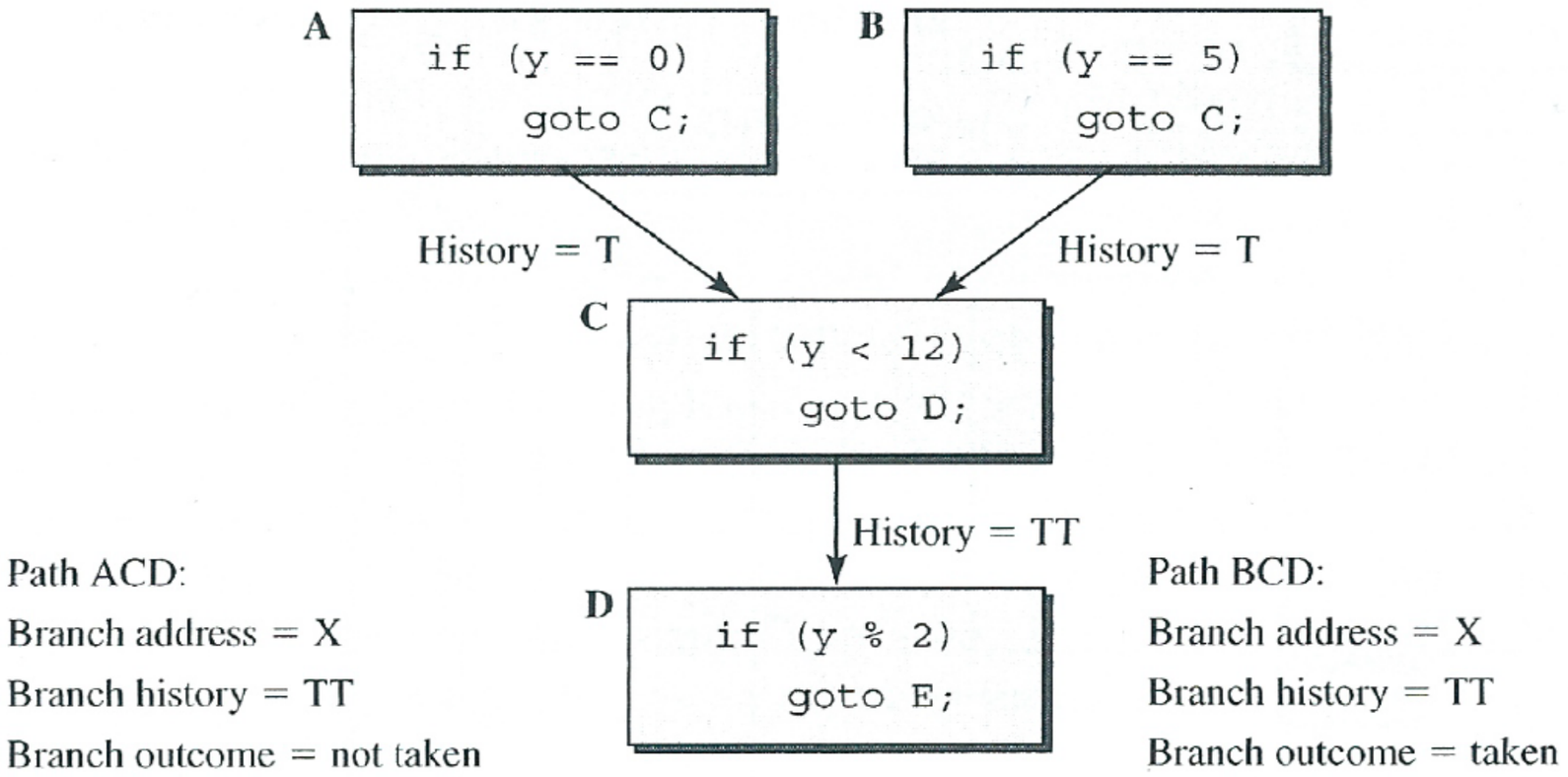
18-740: Modern Computer Architecture & Design

Lecture 3:
"Advanced Branch Prediction Techniques"

A. Basic Dynamic Branch Prediction
B. Two-Level Adaptive Branch Predictors
C. Interference Reducing Predictors
D. Perceptron Based Predictors
E. Hybrid Branch Predictors

Electrical & Computer ENGINEERING

Path History Based Branch Predictor



- Sometimes T/NT history is not enough
- Path history (PC values) can help

The Power of Branch Correlation

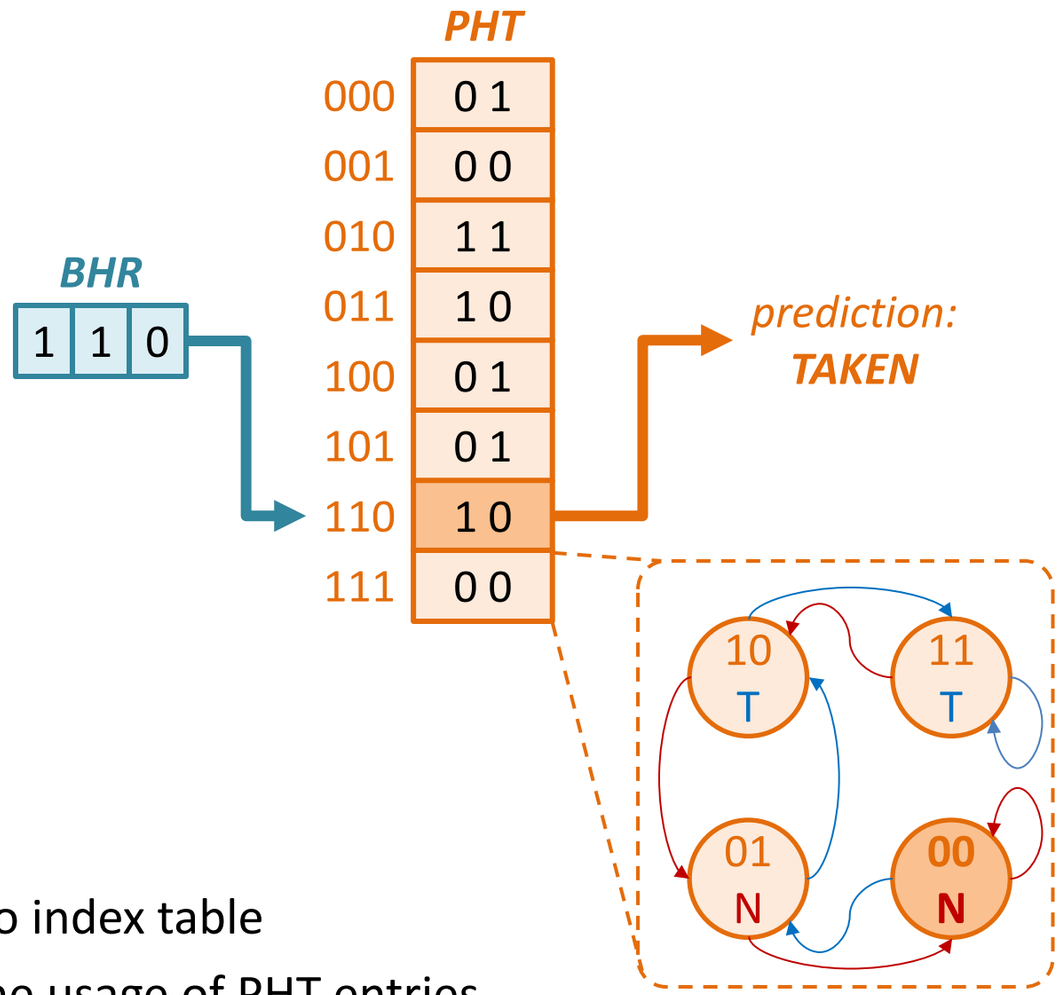
- So far, we have predicted each static branch solely on its own past behavior, and not the behavior of neighboring static branch instructions
- An example of where neighbors can help inform branch decisions:

```
x = 0;
if(someCondition) x = 3;          /* Branch A */
if(anotherCondition) y += 19;     /* Branch B */
if(x <= 0) doSomething();         /* Branch C */
```

- Let's focus on **two types of correlation**
  - **Global:** the decisions of neighboring branch instructions
  - **Local:** the decisions of prior dynamic instances of the same static branch

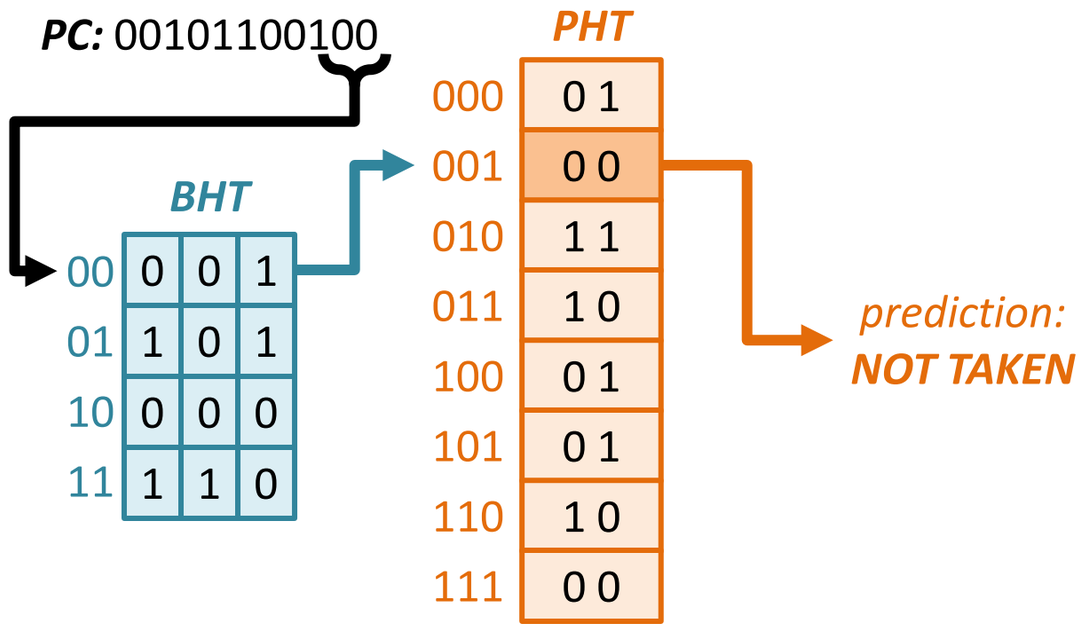
Global History Branch Predictor

- Makes use of a **branch history register** (BHR)
  - A FIFO queue of the last *n* branch decisions made by *any* branch instruction (here, *n* = 3)
  - 1 = taken, 0 = not taken
- Use the current BHR value (i.e., pattern) to index the **pattern history table** (PHT)
- Each PHT entry acts like its own predictor (e.g., a two-bit state machine)
- Update the PHT entry based on the actual decision first, then update the BHR
- **Variant: gshare**
  - XOR BHR with branch address, use XORed value to index table
  - XOR acts as a hash function to better distribute the usage of PHT entries



Local History Branch Predictor

- Instead of having a single BHR, let's replace it with a **branch history table** (BHT)
  - **Use PC to index BHT:** each static branch now has its own history
  - Correlation only within a single static branch instruction
- BHT contents are then used to index the PHT
- Can require more storage than the global history predictor

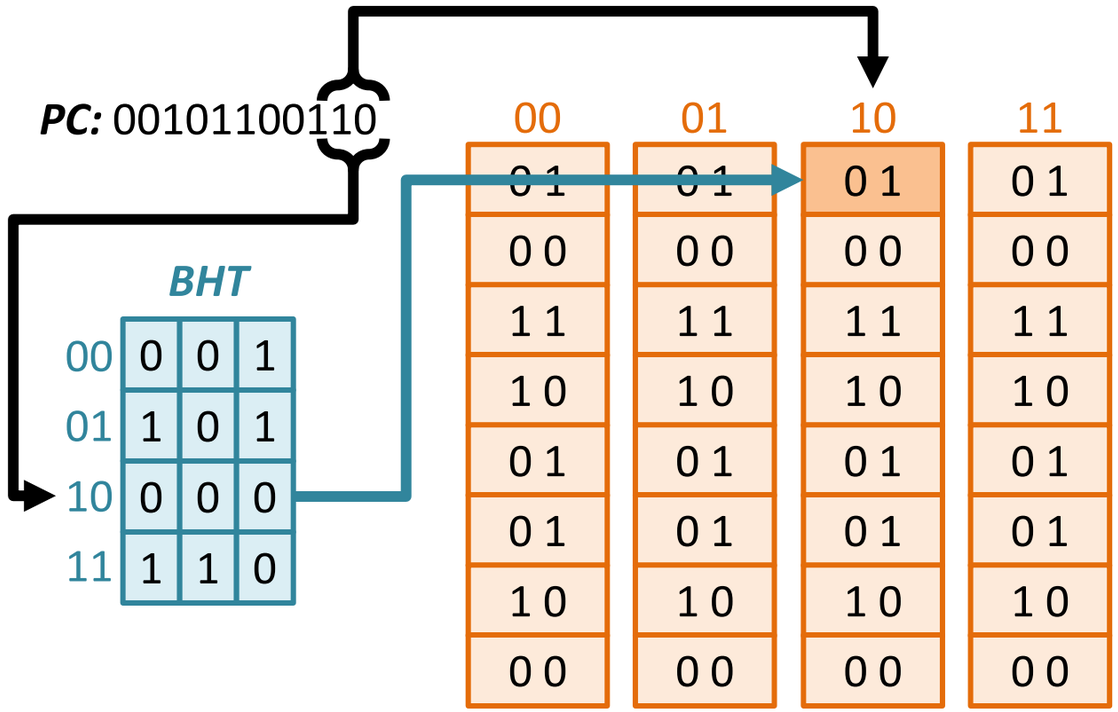


Is this better or worse than the global history branch predictor?

You'll find out in Lab 1!

Two-Level Adaptive Branch Predictor

- Generalization of correlation predictors
- **First level: branch history register(s)**
  - GA: global adaptive
  - PA: per-address adaptive
- **Second level: pattern history table(s)**
  - g: global (i.e., single PHT)
  - s: shared/set (can be indexed using PC, opcode, compiler tag, etc.)
  - p: per-address
- GAg = Global History Predictor on Slide 9
- PAg = Local History Predictor on Slide 10
- At right: PAp – per-address adaptive BHR, per-address PHT

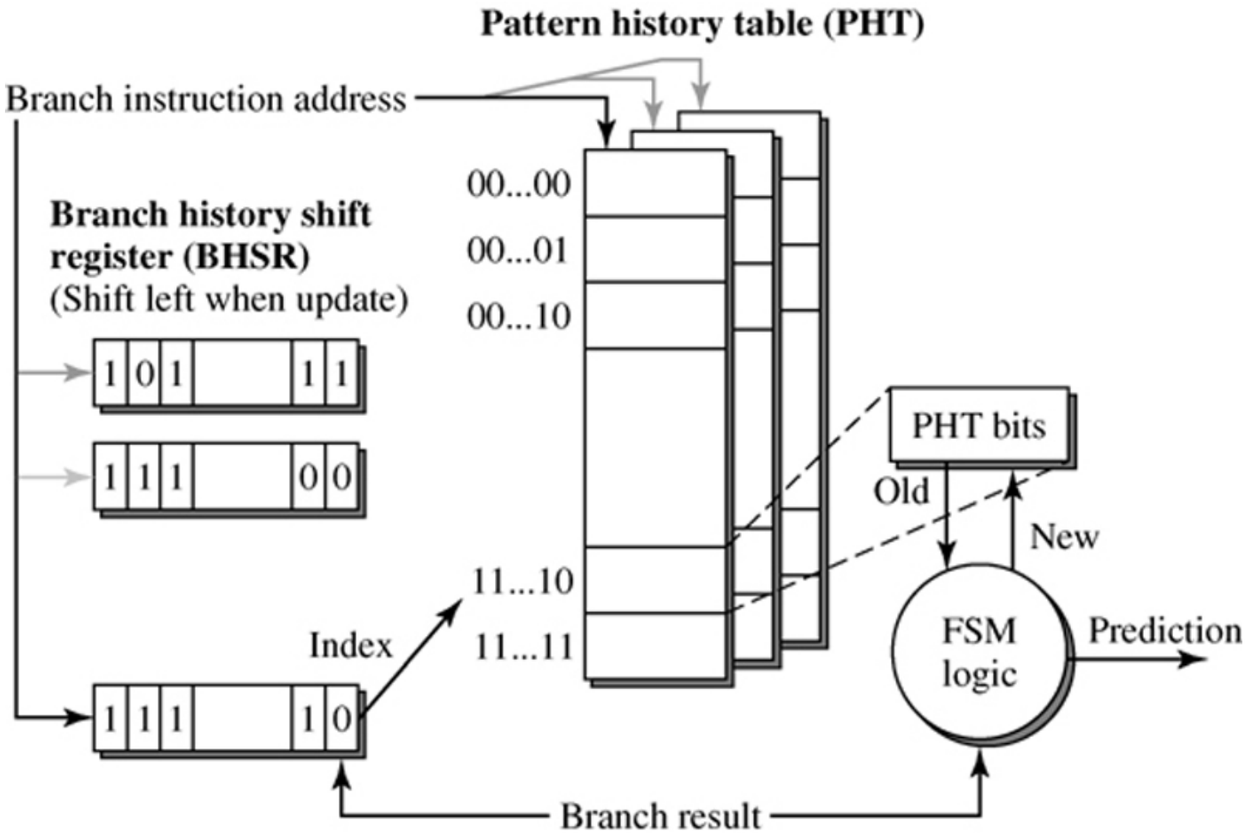


Two-level Adaptive Branch Predictors: Taxonomy

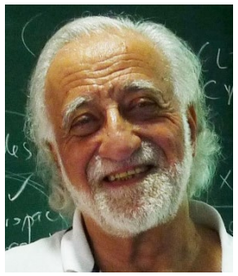
- Based on indices for branch history and pattern history
  - BHR: {G,P}: {Global history, Per-address history}
  - PHT: {g,p,s}: {Global, Per-address, Set}
    - g: use the BHR output as the address into the PHT
    - p: combine the BHR output with some bits from the PC
    - s: use an arbitrary hashing function for PHT addressing
  - 9 combinations: GAg, GAp, GAs, PAg, PAp, PAs, SAg, SAp and SAs
- PAg first described by Yeh and Patt in MICRO 1991
- **Seminal paper: Yeh and Patt, ISCA 1992**
  - Discusses the taxonomy of different two-level predictors
  - Shows comparative results of each type

[T. Yeh and Y. Patt. Two-Level Adaptive Branch Prediction. Intl. Symposium on Microarchitecture, November 1991.]

2-Level Adaptive Branch Prediction [Yeh & Patt, 1991]



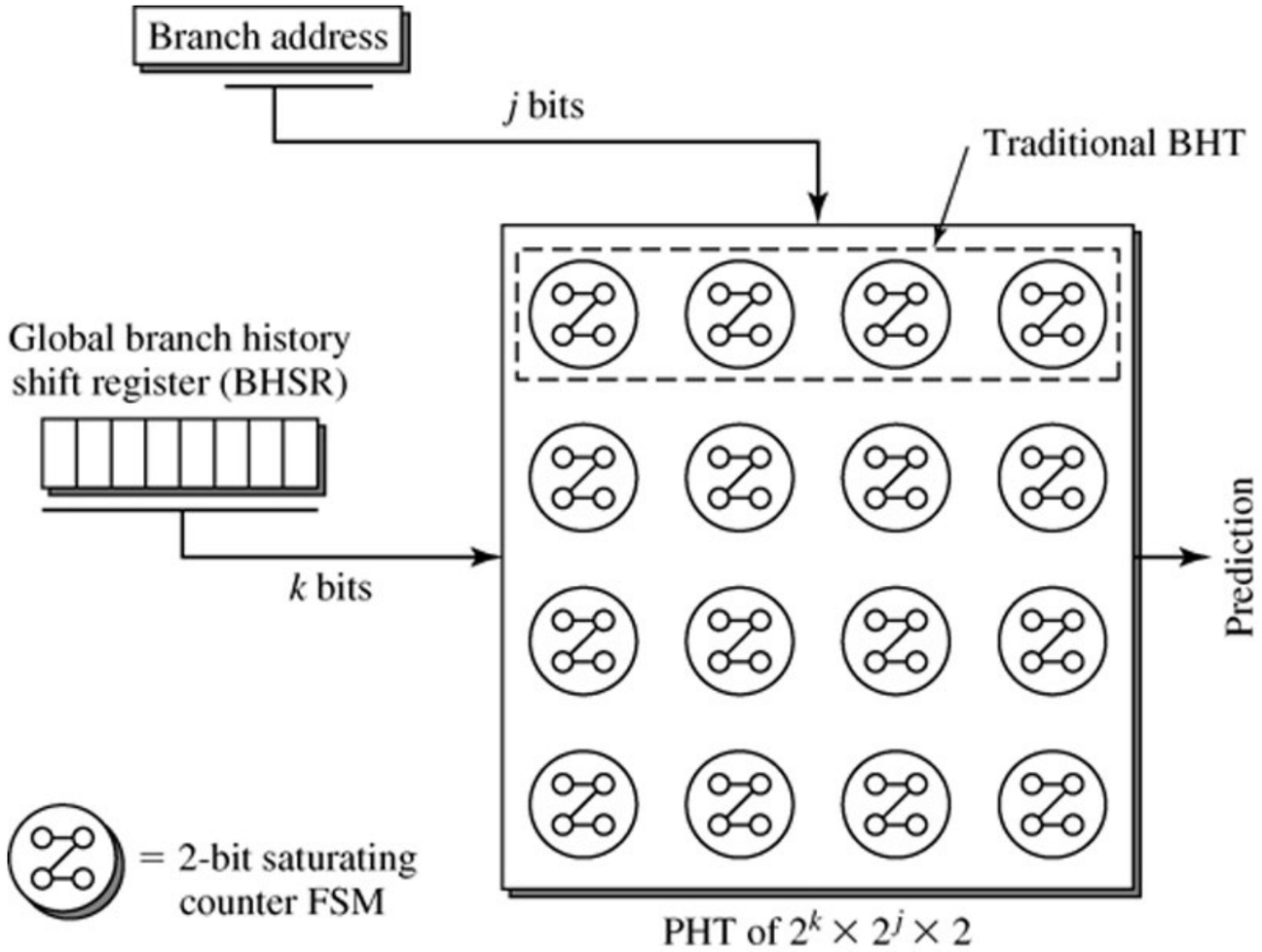

Tse-Yu Yeh


Yale N. Patt

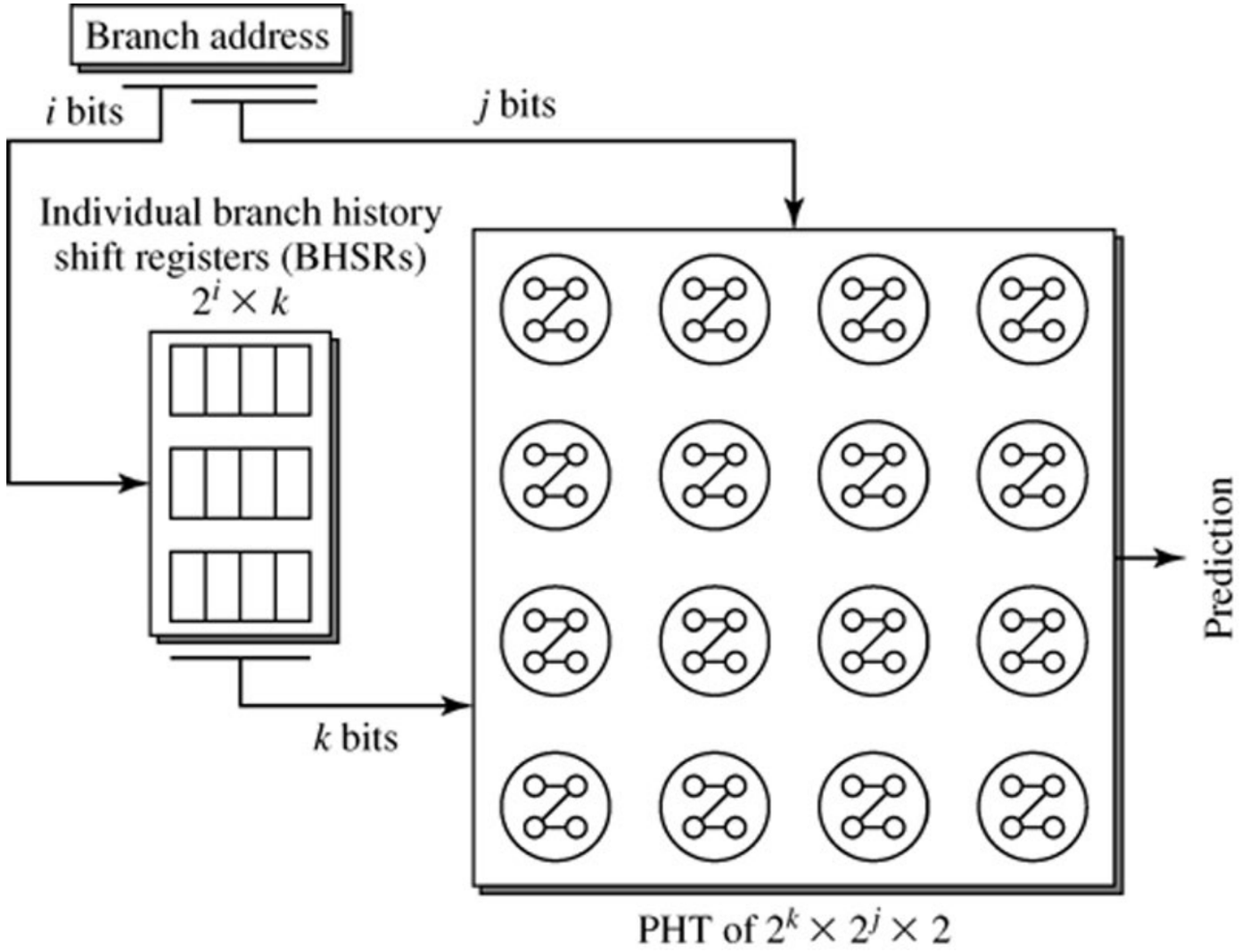
To achieve 97% average prediction accuracy for SPEC 92 benchmarks:

G (1) BHR: 18 bits;	g (1) PHT: 2^{18} x 2 bits	*total = 524 kbits*
P (512x4) BHR: 12 bits;	g (1) PHT: 2^{12} x 2 bits	*total = 33 kbits*
P (512x4) BHR: 6 bits;	s (512) PHT: 2^{6} x 2 bits	*total = 78 kbits*

Example: Global BHSR Scheme (GAs)

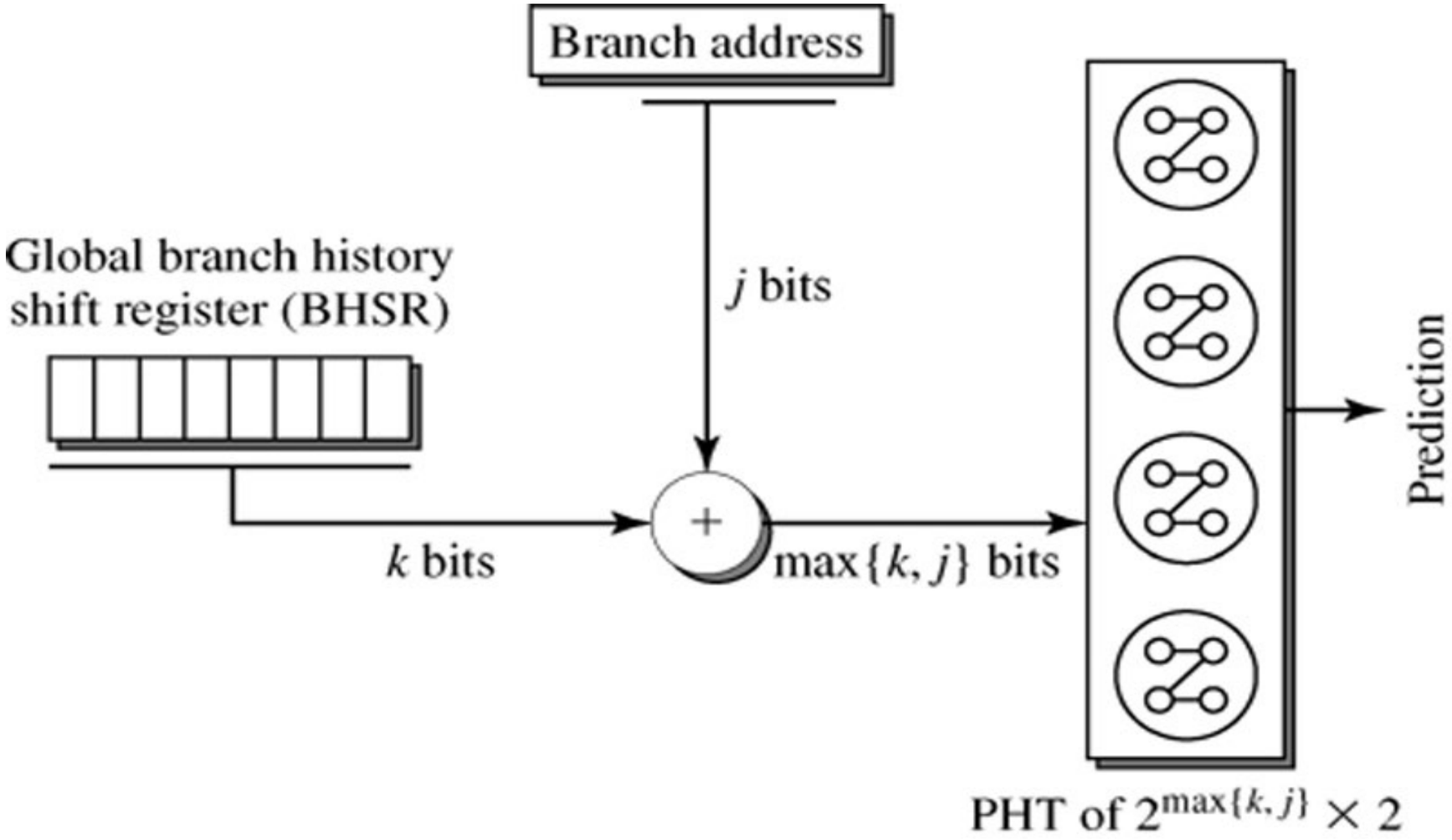


Example: Per-Branch BHSR Scheme (PAs)



Gshare Branch Predictor [Scott McFarling]


Scott McFarling



Causes for Mispredictions

- Fundamentally unpredictable branches
  - Cold miss, data dependent, …
- Training period
  - Need some time to warm up the predictor
  - The more patterns detected, the longer it takes to train
    - E.g. assume a global history predictor with 10 bits of history
    - Need to potentially train up to 2^{10} entries for a specific branch
- Insufficient history to capture patterns
- Aliasing/interference
  - Branch direction predictors have limited capacity and no tags
  - Negative aliasing: two branches train same entry in opposite directions
  - Positive/neutral aliasing: two branches train same entry in same direction

18-740: Modern Computer Architecture & Design

Lecture 3:
"Advanced Branch Prediction Techniques"

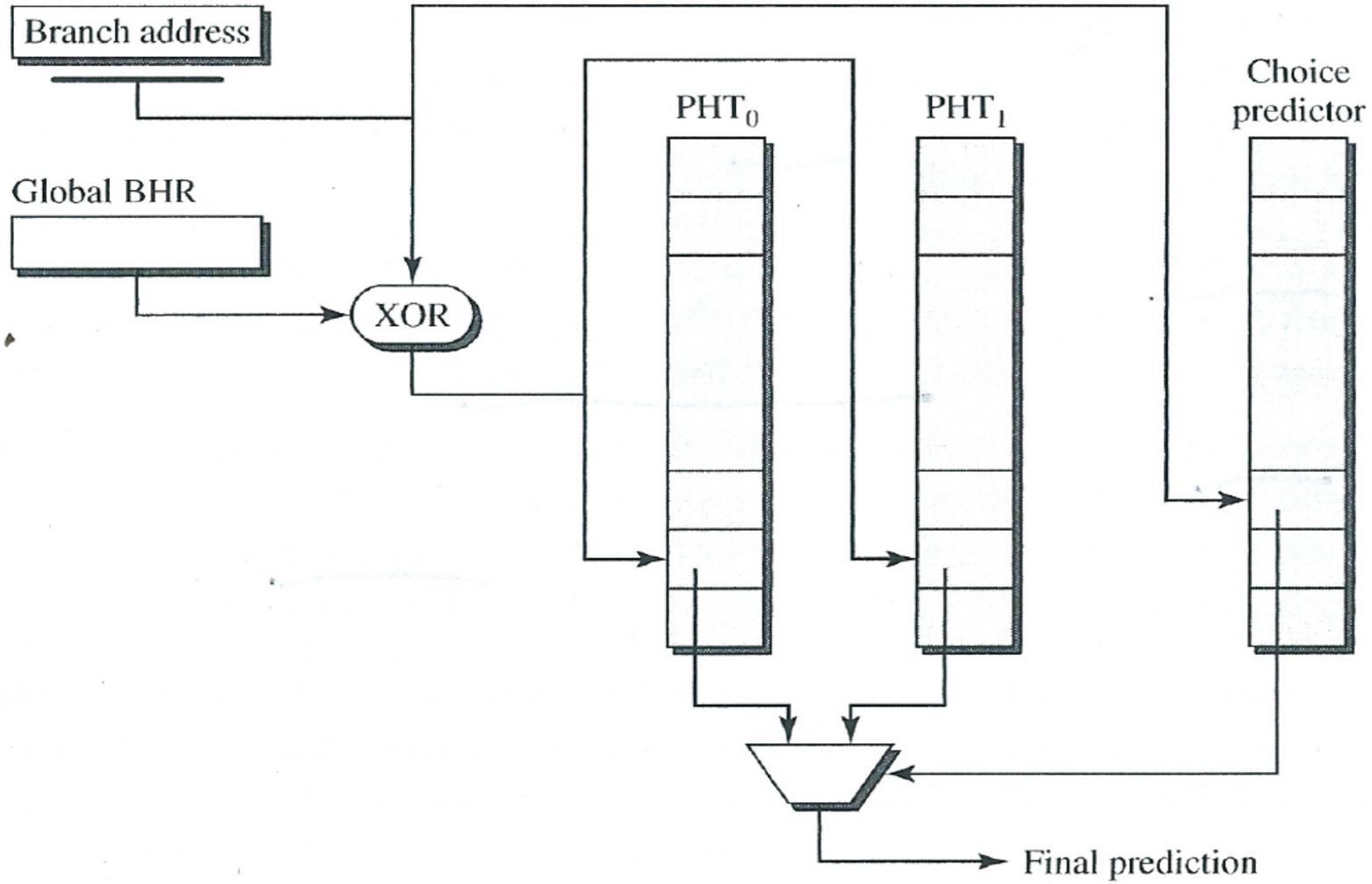
A. Basic Dynamic Branch Prediction
B. Two-Level Adaptive Branch Predictors
C. Interference Reducing Predictors
D. Perceptron Based Predictors
E. Hybrid Branch Predictors

Electrical & Computer ENGINEERING

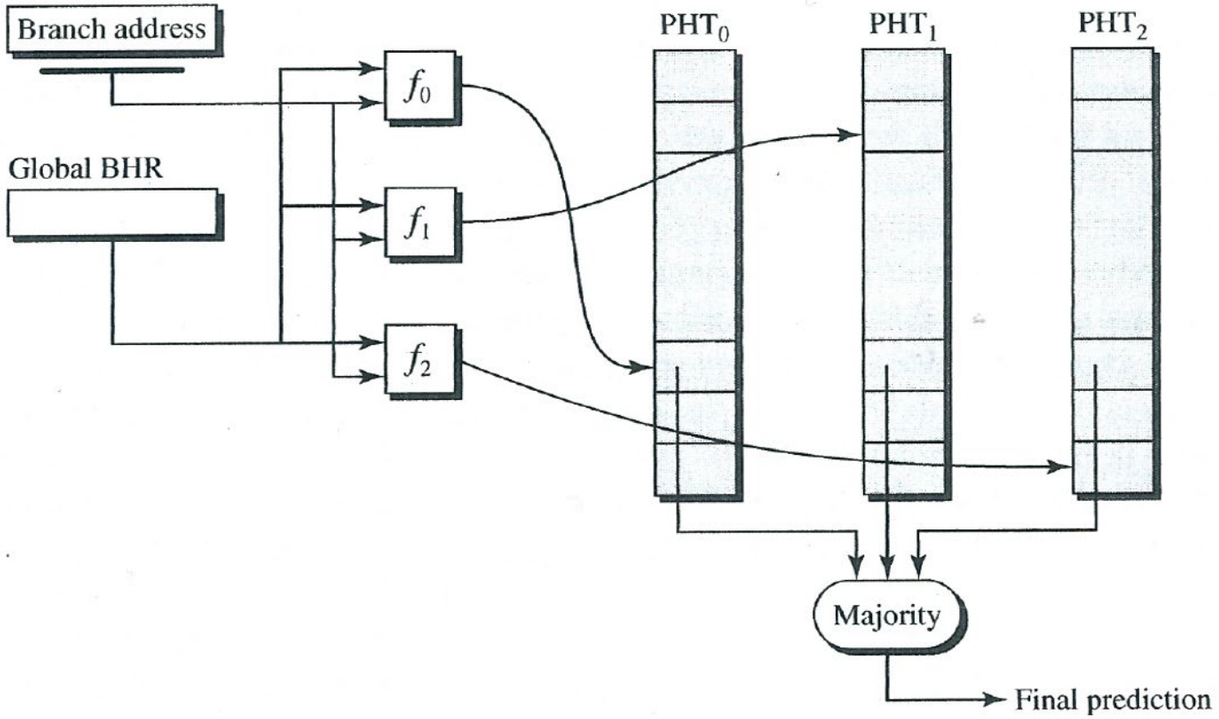
Other Advanced Branch Predictor Techniques

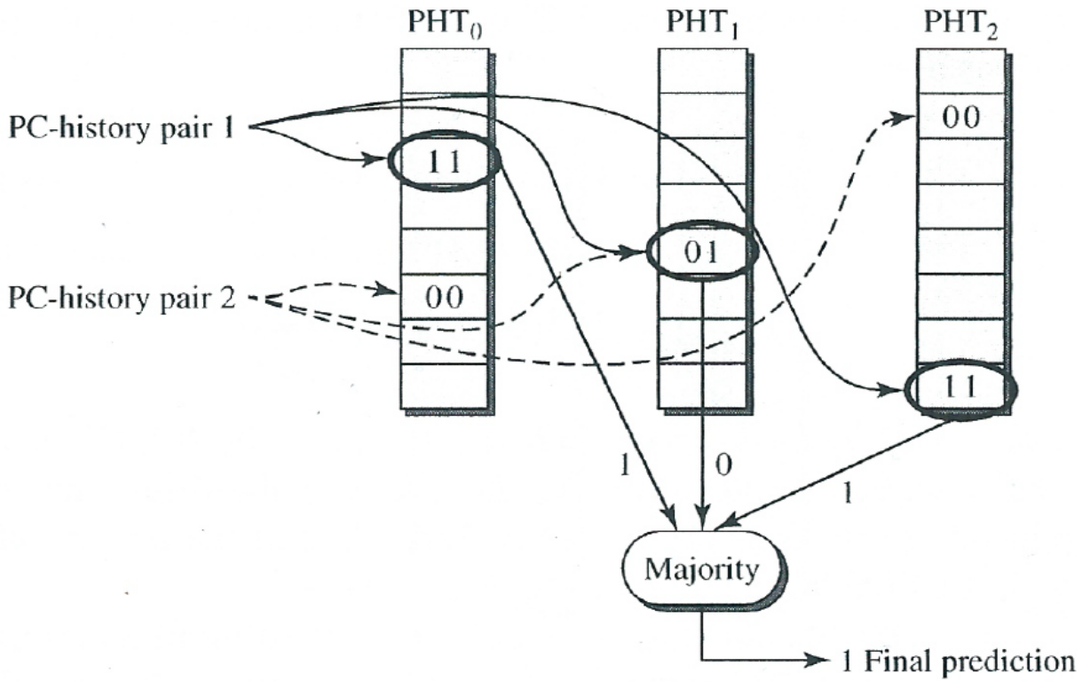
- **Bi-Mode Predictor**
  - Separate PHT for biased (i.e., mostly taken/mostly non-taken) branches (predictor to identify these)
  - Eliminates negative aliasing for branches with a more balanced history
- **G-Skewed Predictor** (used in Alpha EV8)
  - Use multiple hash functions into PHTs & vote on outcome
  - Reduce chance of negative interference affecting prediction
- **Agree Predictor**
  - BTB gives you a basic prediction, extra PHT tells you if you should agree with the BTB
  - Biased branches have positive interference regardless of direction…
- **YAGS** (Yet Another Global Scheme)
  - Keep a small tagged cache with branches that experience interference
- Many more: Branch filtering, selective branch inversion, alloyed history predictors, path history predictors, variable path length predictors, dynamic history length fitting predictors, loop counting predictors, data-flow predictors, analog circuit predictors, …

Bi-Mode Branch Predictor



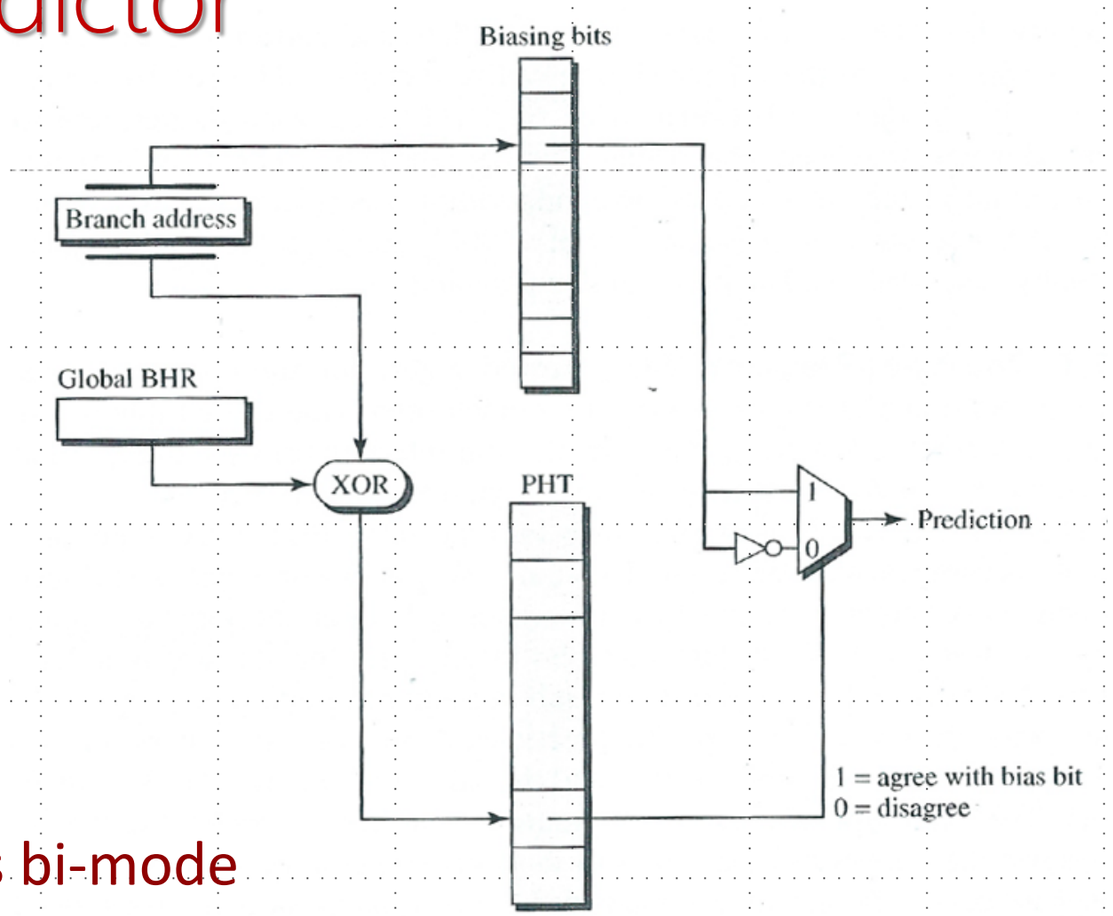
G-Skewed Branch Predictor





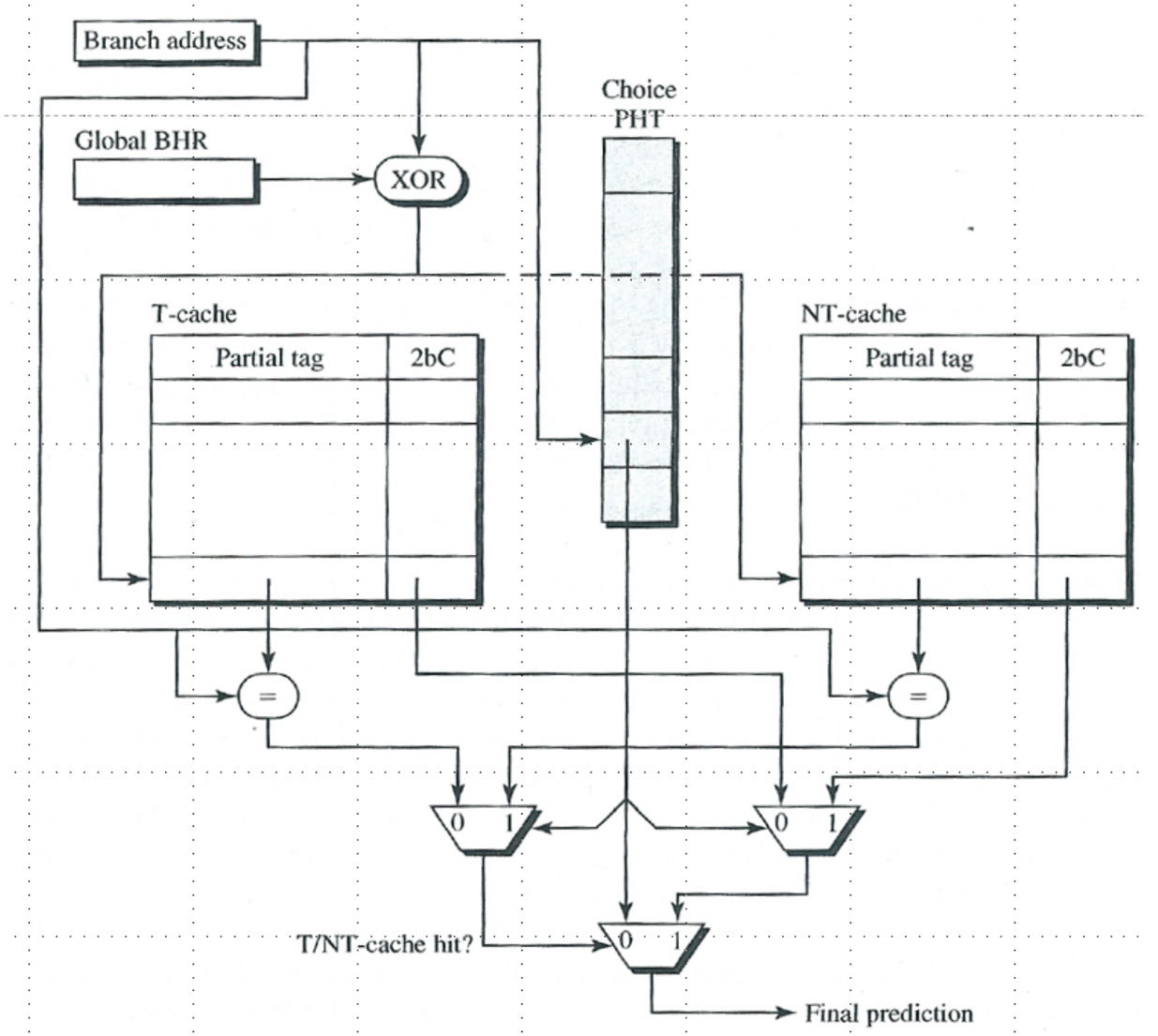
- Multiple PHT banks indexed by different hash functions
  - Conflicting branch pair unlikely to conflict in more than one PHT
- Majority vote determines prediction
- Used in the cancelled Alpha 21464

Agree Branch Predictor



- Same principle as bi-mode
- PHT records whether branch bias matches outcome
  - Exploits 70-80% static predictability
- Used in in HP PA-8700

YAGS Branch Predictor



18-740: Modern Computer Architecture & Design

Lecture 3:
"Advanced Branch Prediction Techniques"

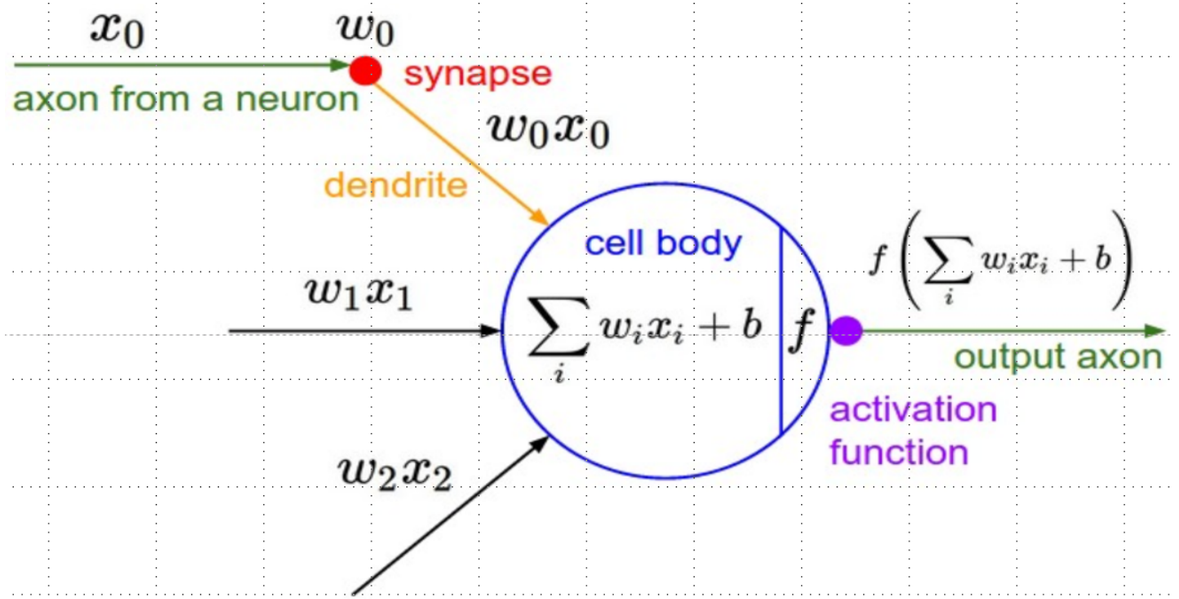
A. Basic Dynamic Branch Prediction
B. Two-Level Adaptive Branch Predictors
C. Interference Reducing Predictors
D. Perceptron Based Predictors
E. Hybrid Branch Predictors

Electrical & Computer ENGINEERING

Perceptron Based Branch Prediction

[Jimenez, Lin, HPCA 2001]

- Perceptron
  - Basis in AI concept [1962]
  - Computes Boolean result based on multiple weighted inputs
- Adapted for branch prediction
  - x_i from branch history (1 T, -1 NT)
  - w_i incremented whenever branch outcome matches xi
  - Finds correlation between current branch and <u>any subset of</u> prior branches



Another diagram, showing the biological inspiration. The **activation function** is what people now call the non-linear function applied to the weighted input sum to produce the output of the artificial neuron - in the case of Rosenblatt's Perceptron, the function just a thresholding operation.

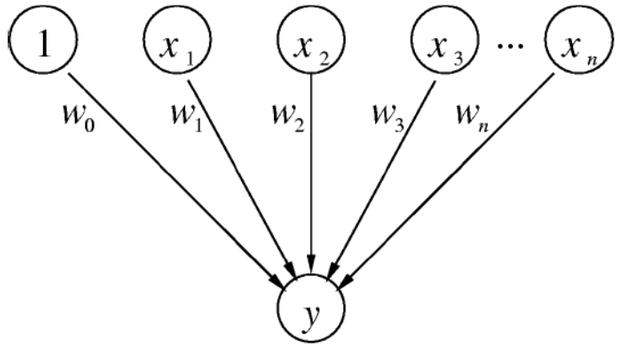
$$y = w_0 + \sum_{i=1}^{n} x_i w_i$$

Perceptrons: A Primer

- Computed using the equation

$$y = w_0 + \sum_{i=1}^{n} x_i w_i$$

  - y: outcome — ***branch prediction***
  - w_0: bias weight — ***branch bias***
  - w_i: weight — ***branch importance***
  - x_i: input — ***branch history***



- Training algorithm

t is the Boolean branch outcome.
θ is the training threshold.

```
if |y| ≤ θ or ((y ≥ 0) ≠ t) then
    for each 0 ≤ i ≤ n in parallel
        if t = x_i then
            w_i := w_i + 1
        else
            w_i := w_i − 1
        end if
    end for
end if
```

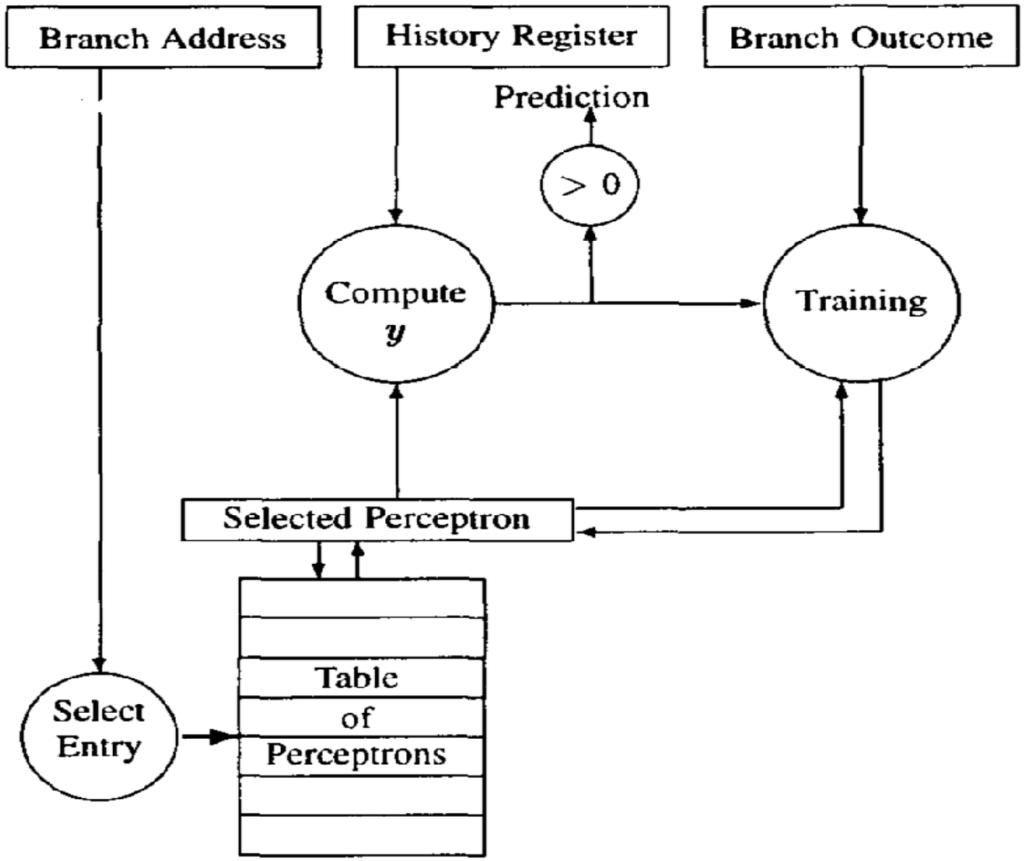
D. Jiménez, "Perceptron for Dummies", JILP CBP, 2004.

Perceptron Implementation

- Complex dot product must be computed for every prediction
  - Too slow
- Arithmetic tricks, pipelining:
  - Daniel A. Jimenez and Calvin Lin. Neural methods for dynamic branch prediction. ACM Transactions on Computer Systems, 20(4):369–397, November 2002.
  - Analog circuit implementation also possible
- Key insight:
  - Not all branches in history are important (correlate)
  - Perceptron weights learn this

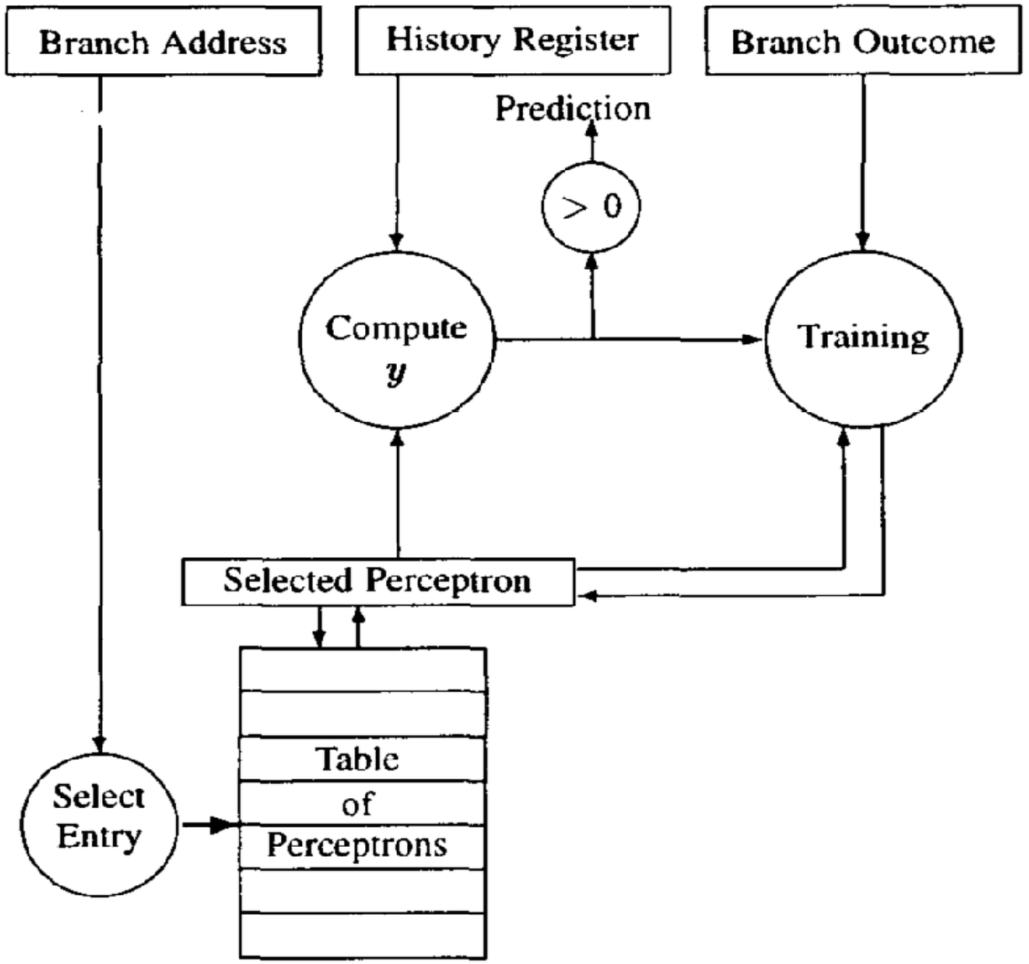

Daniel Jiménez


Calvin Lin



Perceptron Branch Predictor

- Based on AI concept of **perceptrons**
- Uses multiple weighted inputs to make a binary decision
  - > 0: taken; ≤ 0: not taken
  - **Weights learn which of the prior branches are *not* important**
- Early use of AI/ML in architecture HW: Jiménez and Lin, HPCA 2001
- Now starting to appear in commercial processors, e.g.,
  - AMD Ryzen
  - Samsung Exynos



18-740: Modern Computer Architecture & Design

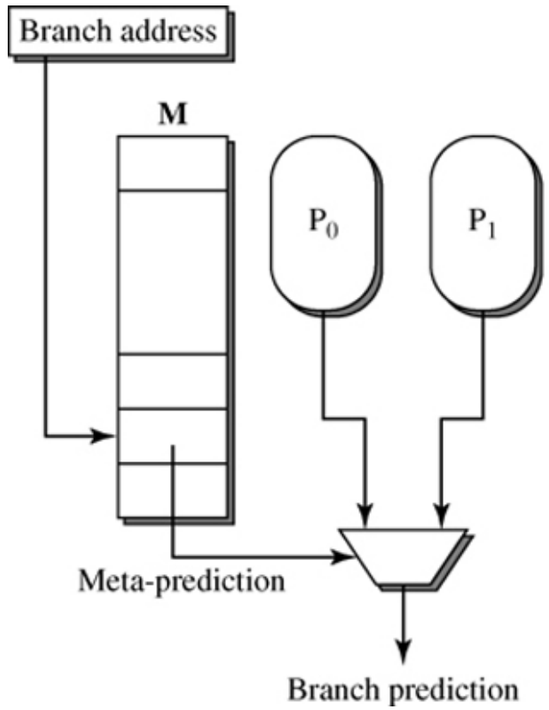
Lecture 3:
"Advanced Branch Prediction Techniques"

A. Basic Dynamic Branch Prediction
B. Two-Level Adaptive Branch Predictors
C. Interference Reducing Predictors
D. Perceptron Based Predictors
E. Hybrid Branch Predictors

Electrical & Computer ENGINEERING

Combining (a.k.a Hybrid/Tournament) Predictor

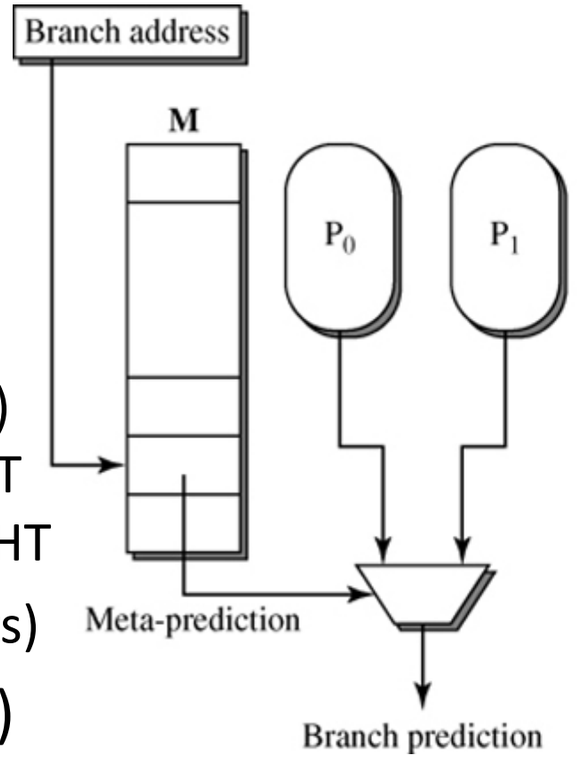
- What if different programs exhibit different patterns? **No one predictor is best**
- **Combining (or hybrid or tournament) predictors: use multiple predictors and choose one**
  - Each predictor (e.g., P_0, P_1) tries to capture a particular program behavior
  - **Meta-predictor:** another history-based prediction scheme to "predict" which predictor should be used for a particular branch (there can only be one victor)
  - Best of all worlds – this works quite well

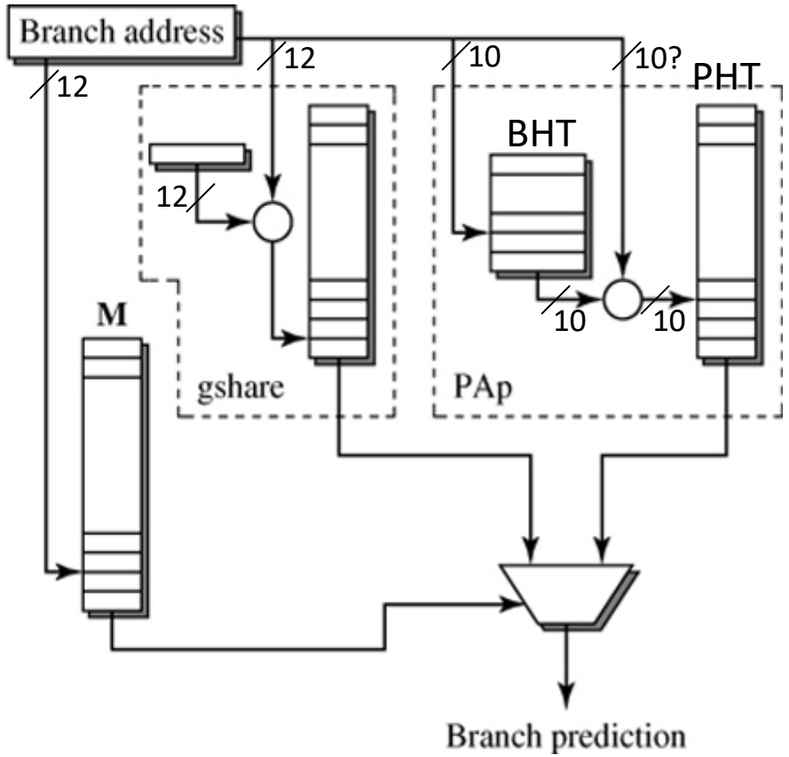


- Original paper: McFarling, DEC tech report, 1993

Tournament Branch Prediction in the Alpha 21264

- Alpha: family of aggressive processors by Digital (DEC) in the 1990s
- Predictor 0
  - Gshare
  - 12 bits of history (4K counters)
- Predictor 1
  - PAp
  - BHT: 1K entries (10 br. addr. bits) 10-bit local history from the BHT entry is used to index into the PHT
  - PHT: 1K entries (10 local hist. bits)
- Meta-predictor (a.k.a. selector)
  - 4K entry table of state machines





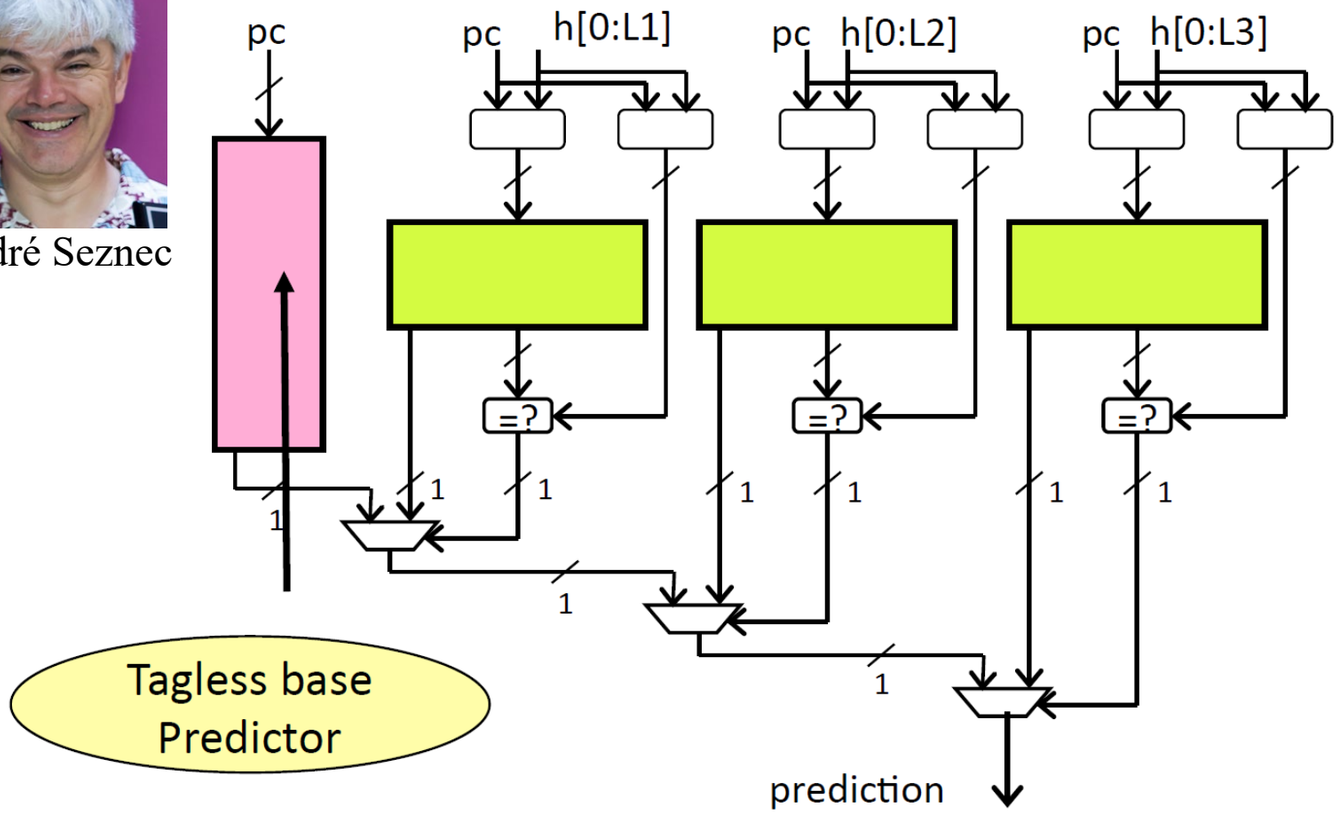
Scott McFarling. Combining Branch Predictors. TN-36, DEC Western Research Laboratory, June 1993.

[André Seznec, INRIA/IRISA, MICRO'11, December 3-7, 2011, Porto Alegre, Brazil]

The TAGE Branch Predictor [MICRO 2011]



André Seznec



Championship Branch Prediction

in conjunction with:
ISCA-38 http://isca2011.umaine.edu/

Branch Prediction Is Not A Solved Problem: Measurements, Opportunities, and Future Directions

Chit-Kwan Lin and Stephen J. Tarsa
Intel Corporation
Santa Clara, CA
{chit-kwan.lin, stephen.j.tarsa}@intel.com

arXiv:1906.08170v1 [cs.DC] 13 Jun 2019

***Abstract*—Modern branch predictors predict the vast majority of conditional branch instructions with near-perfect accuracy, allowing superscalar, out-of-order processors to maximize speculative efficiency and thus performance. However, this impressive overall effectiveness belies a substantial missed opportunity in single-threaded instructions per cycle (IPC). For example, we show that correcting the mispredictions made by the state-of-the-art TAGE-SC-L branch predictor on SPECint 2017 would improve IPC by margins similar to an advance in process technology node.**

In this work, we measure and characterize these mispredictions. We find that they categorically arise from either (1) a small number of systematically hard-to-predict (H2P) branches; or (2) rare branches with low dynamic execution counts. Using data from SPECint 2017 and additional large code footprint applications, we quantify the occurrence and IPC impact of these two categories. We then demonstrate that increasing the resources afforded to existing branch predictors does not alone address the root causes of most mispredictions. This leads us to reexamine basic assumptions in branch prediction and to propose new research directions that, for example, deploy machine learning to improve pattern matching for H2Ps, and use on-chip phase learning to track long-term statistics for rare branches.

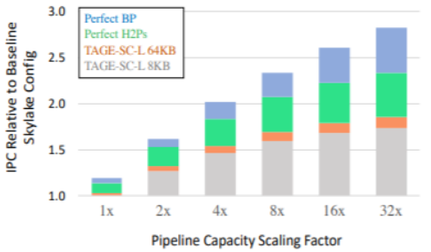


Fig. 1: Without better branch prediction, scaling the pipeline capacity of an Intel Skylake configuration will produce diminishing returns in single-threaded IPC for SPECint 2017.

I. Introduction

Branch prediction is critical to the performance of modern superscalar processors [1], [2], [3] and is implemented in dedicated branch prediction units (BPUs). BPUs work by training statistical models of branch directions observed as instructions are retired, and then using these models to predict unresolved directions for subsequent branches as they are fetched. BPU predictions drive speculative execution, a key technique for hiding latency in out-of-order CPUs.

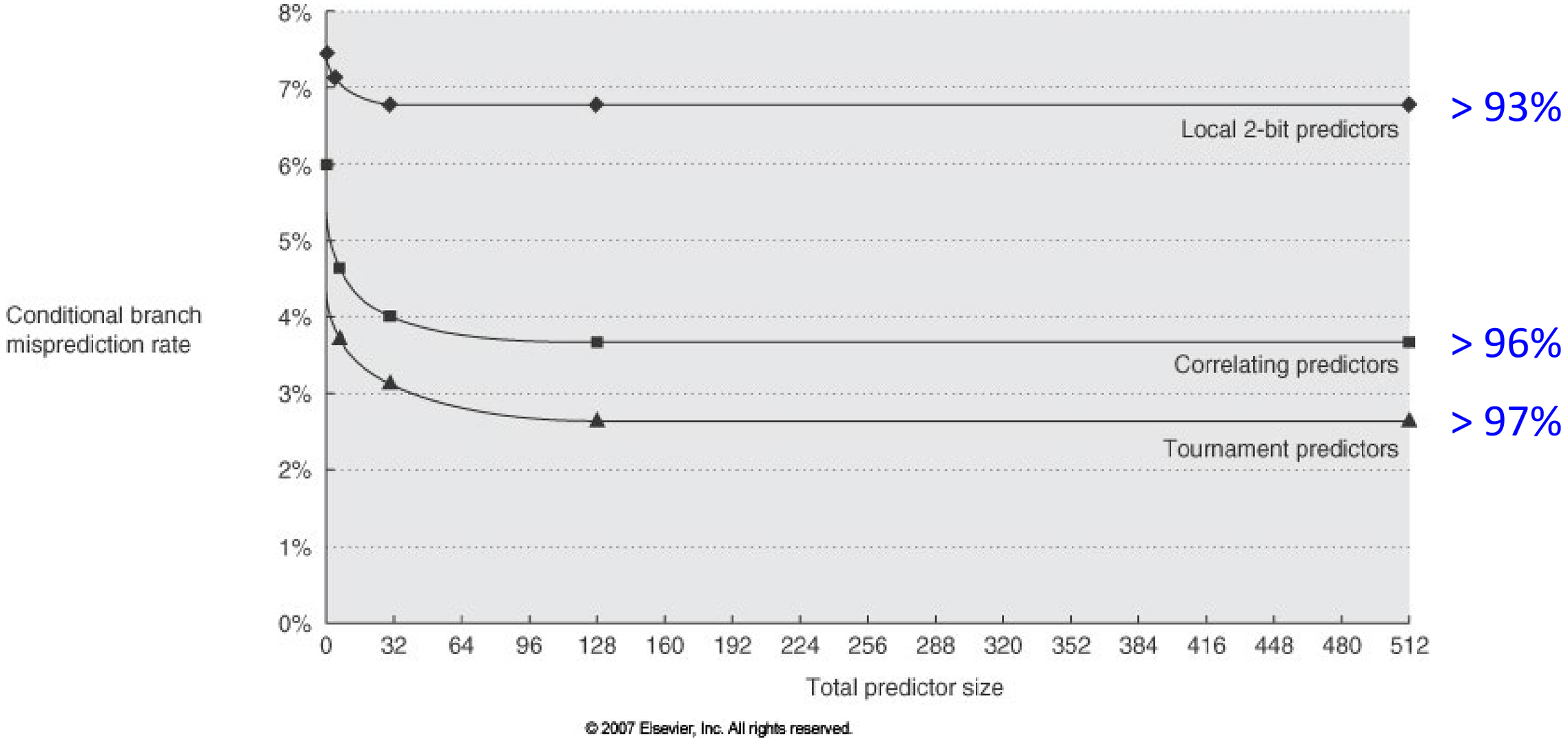
Though state-of-the-art branch predictors achieve near-perfect prediction accuracy on the vast majority of static branches, substantial performance gains can be unlocked by correcting their remaining mispredictions. Mispredictions delay subsequent instructions, trigger instruction pipeline flushes, and reduce speculation efficiency. For example, Fig. 1 shows single-threaded performance for the SPECint 2017 benchmarks on an execution pipeline based on Intel Skylake, as we simulate future designs with increased pipeline capacity (i.e., fetch, decode, execution, load/store buffer, ROB, scheduler, and retire resources) in the ChampSim simulator [4], [5]. Using the TAGE-SC-L 8KB branch predictor [6], mispredictions represent an 18.5% instructions per cycle (IPC) opportunity at baseline (1x scaling). This gain grows with pipeline scale, e.g., to 55.3% at 4x scaling, a magnitude on par with advancing to the next process technology node.

Microarchitectural advances in branch prediction are thus a source of large potential IPC gains. However, we find only marginal improvements from straightforward scaling of the resources provided to existing predictors. Fig. 1 illustrates this point: increasing TAGE-SC-L storage eight-fold to 64KB returns just 2.7% additional IPC in a best-case scenario where no additional prediction latency is incurred.

In this paper, we perform a deep dive into the causes of these mispredictions, and demonstrate that fundamentally new approaches to branch prediction are needed to address them. We identify two primary issues: (1) systematically hard-to-predict (H2P) branches; and (2) rare branches with low dynamic execution counts over tens of millions of instructions. In addition to degrading IPC, these branches trigger inefficient consumption of BPU storage on a large scale, and exhibit temporal behaviors inconsistent with the short-term statistics emphasized in known algorithms. We propose new approaches that deploy powerful machine learning models to improve the pattern matching that drives H2P predictions (e.g. using low-precision convolutional neural networks, as developed in our companion paper [7]), as well as on-chip phase recognition to capture long term predictive statistics.

https://arxiv.org/pdf/1906.08170.pdf

Are We Done with Branch Predictors?...Why Not?

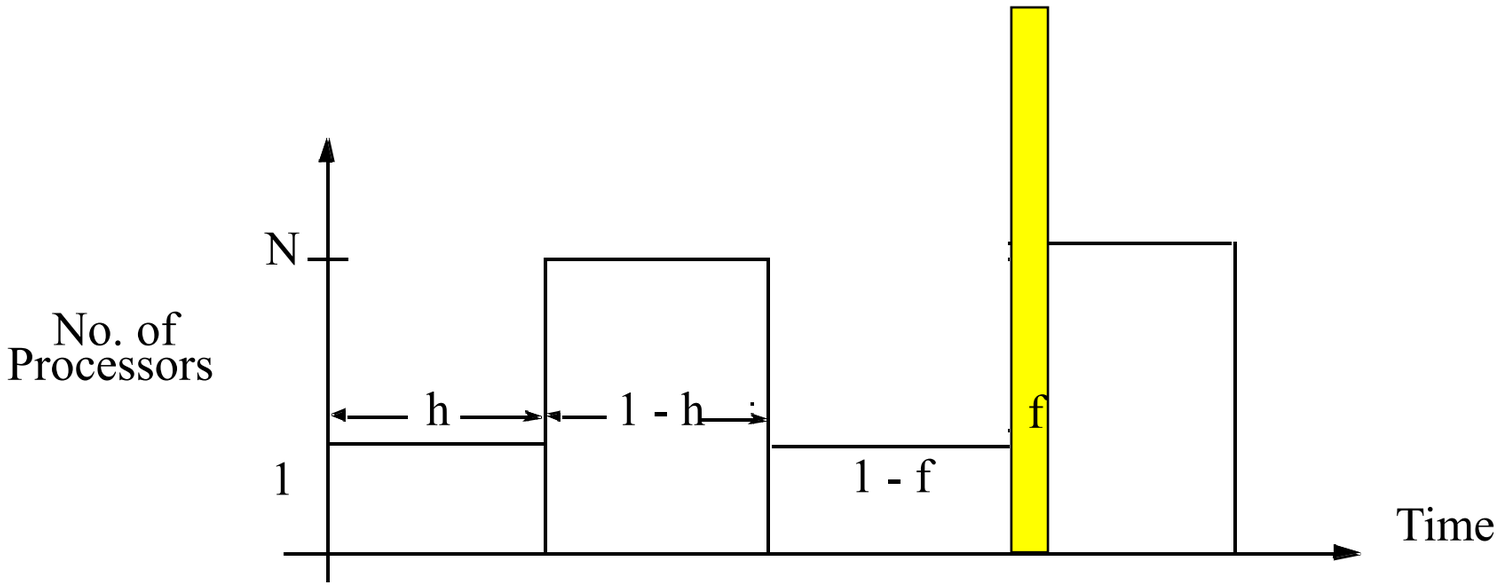


© 2007 Elsevier, Inc. All rights reserved.

Revisit Amdahl's Law

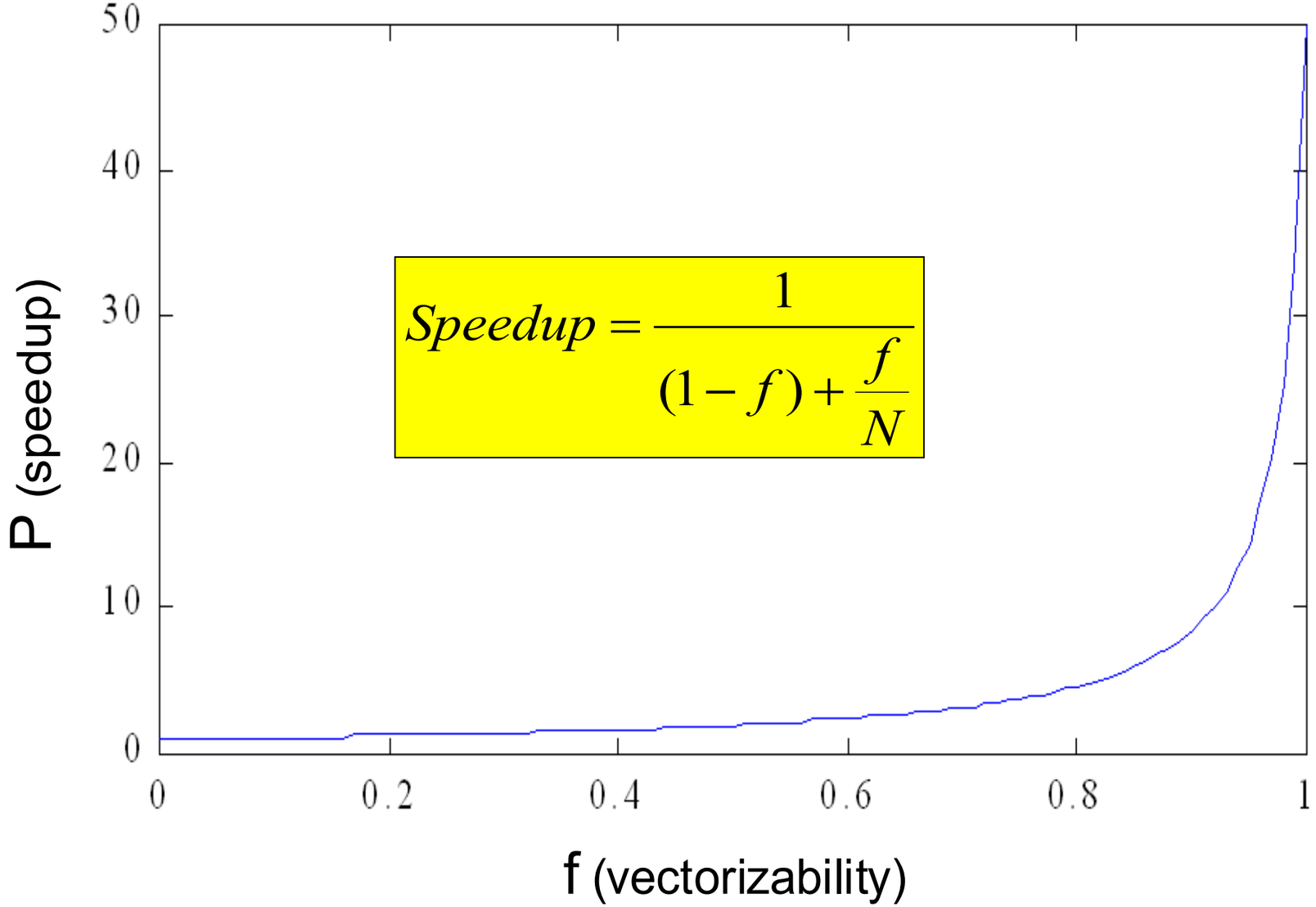
- Sequential bottleneck
- Even if *N* is infinite
  - Performance limited by non-vectorizable portion (1 – *f*)

$$\lim_{N\to\infty} \frac{1}{(1-f)+\frac{f}{N}} = \frac{1}{1-f}$$





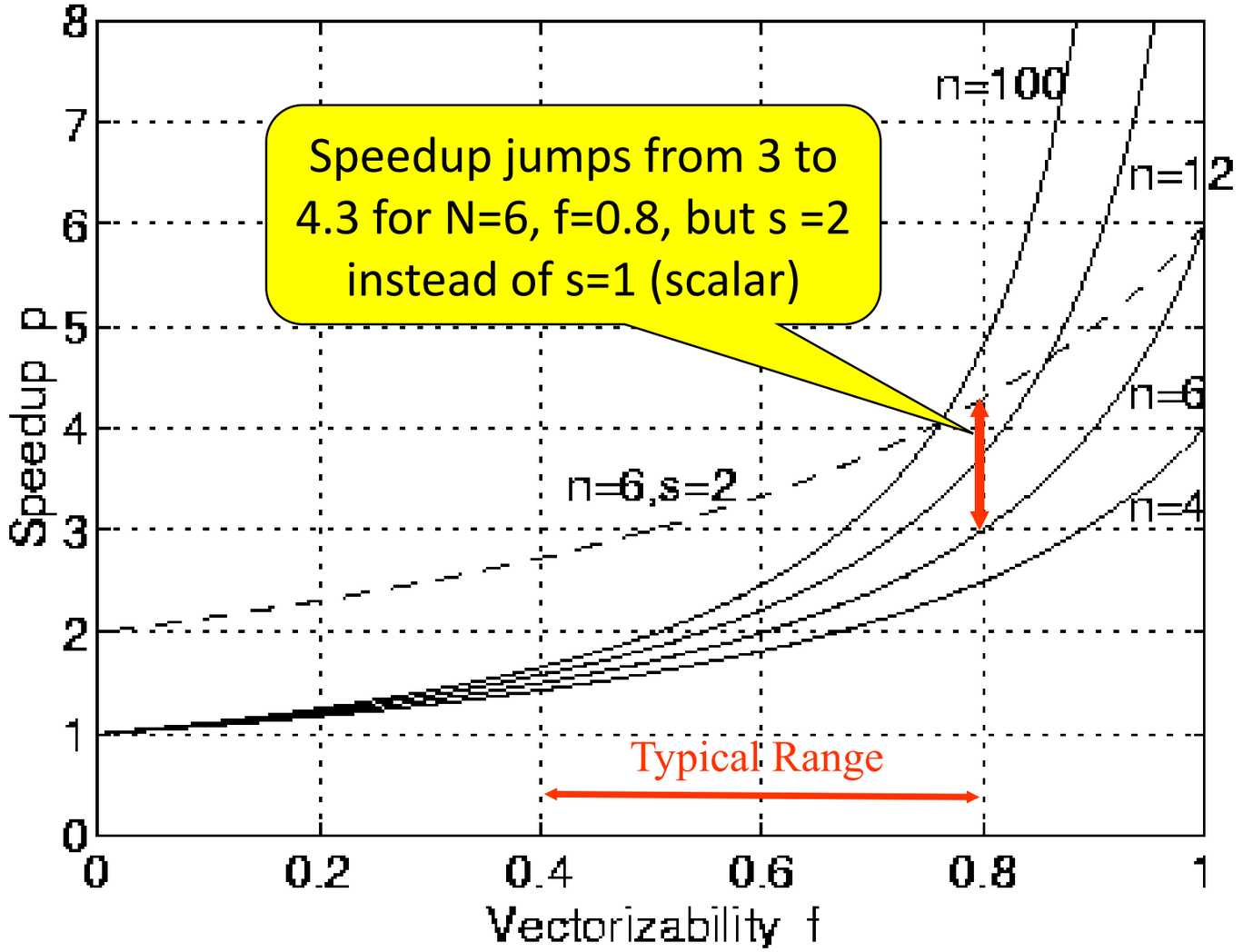
"Tyranny of Amdahl's Law" [Bob Colwell, CMU-Intel-DARPA]



- Suppose that a computation has a 4% serial portion, what is the limit of speedup on 16 processors?
  - 1/((0.04) + (0.96/16)) = 10
  - What is the maximum speedup?
    - 1/0.04 = 25 (with N → ∞)

[Tilak Agerwala and John Cocke, 1987]

Motivation for Superscalar Design



Superscalar Proposal and Motivation

- **Moderate the tyranny of Amdahl's Law**
  - Ease the sequential bottleneck
  - More generally applicable
  - Robust (less sensitive to *f*)
  - Revised Amdahl's Law:

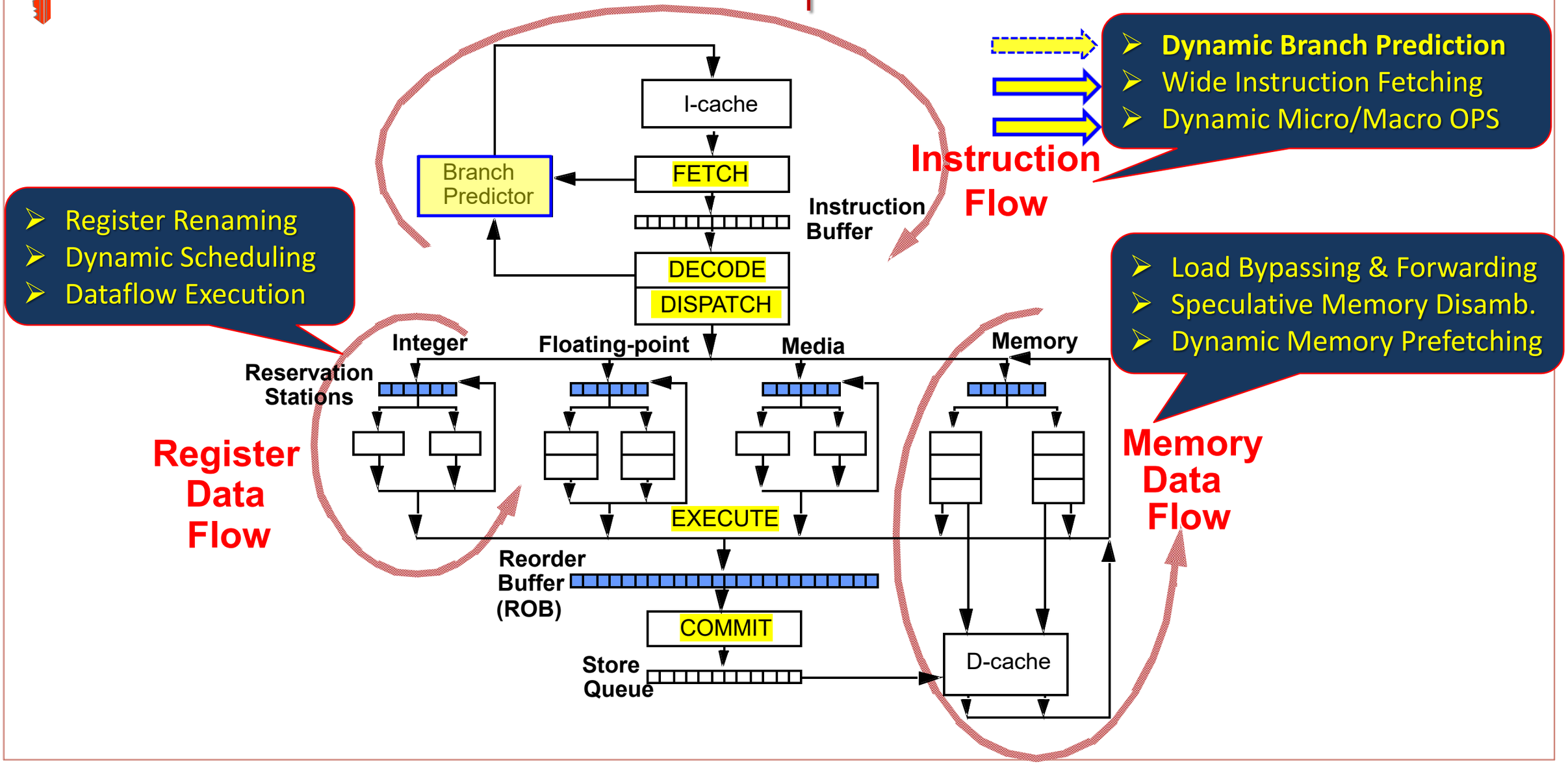
$$Speedup = \frac{1}{\frac{(1-f)}{S} + \frac{f}{N}}$$

Limits on Instruction-Level Parallelism (ILP)

Weiss and Smith [1984]	1.58
Sohi and Vajapeyam [1987]	1.81
Tjaden and Flynn [1970]	1.86 (Flynn's bottleneck)
Tjaden and Flynn [1973]	1.96
Uht [1986]	2.00
Smith et al. [1989]	2.00
Jouppi and Wall [1988]	2.40
Johnson [1991]	2.50
Acosta et al. [1986]	2.79
Wedig [1982]	3.00
Butler et al. [1991]	5.8
Melvin and Patt [1991]	6
Wall [1991]	7 (Jouppi disagreed)
Kuck et al. [1972]	8
Riseman and Foster [1972]	51 (no control dependences)
Nicolau and Fisher [1984]	90 (Fisher's optimism)

[Mike Johnson, *Superscalar Microprocessor Design*, Prentice-Hall, 1991]

Three Flow Paths of Superscalar Processors



18-740: Modern Computer Architecture & Design

Lecture 4:
"Wide Instruction Fetch and Dynamic Decode"

John Paul Shen
September 3, 2026

Next Time...

- Required Reading Assignments:
  - **Chapter 4 and Chapter 5 of Shen and Lipasti (SnL)**
- Recommended Reference:
  - Eric Rotenberg, S. Bennett, and James E. Smith. *"Trace Cache: A Low Latency Approach to High Bandwidth Instruction Fetching."* MICRO, 1996.



Electrical & Computer ENGINEERING